LLM-Based Automatic Unit Test Generation Using Focal-Method

Proposal

Professor: Dr. Karim Elish
Submission Date: February 12, 2026

Team Members::

Siwani Sah
Saarupya Sunkara
Alexandra Hyatali

Florida Polytechnic University

1 Problem Statement

1.1 What problem are we addressing?

Modern software development heavily relies on unit testing to ensure code reliability and maintainability. However, the manual creation of high-quality unit tests remains a time-consuming and labor-intensive process. Although Large Language Models (LLMs) have demonstrated potential in automating code generation, they frequently exhibit limitations when generating unit tests for specific "focal methods" in isolation. Key challenges include:

- **Overgeneration:** The generation of tests that reference non-existent dependencies or validate behaviors beyond the scope of the method.
- **Context Insensitivity:** A failure to comprehend class-level or project-level dependencies necessary for the method's execution.
- **Lack of Alignment:** The production of assertions that do not accurately reflect the original developer's intent or fail to cover critical edge cases.

1.2 Why it matters

Addressing these limitations is critical for:

- **Technical Impact:** Automating the generation of boilerplate tests allows developers to allocate more time to complex integration logic and architectural design.
- **Practical Impact:** Enhancing the compilability and correctness of automatically generated tests increases the reliability and adoption of AI-assisted software engineering tools.

1.3 Who benefits

This research primarily benefits **software developers** by providing a robust tool to accelerate the test-driven development workflow. Furthermore, it contributes to the **software engineering research community** by providing empirical data on the effectiveness of context-aware prompting strategies for LLMs.

2 Research Questions or Goals

Our primary objective is to develop a tool capable of generating high-quality unit tests for a given focal method and to empirically evaluate its performance against developer-written baselines. We propose the following Research Questions (RQs):

- **RQ1:** How does the inclusion of focal-method context (e.g., class signatures, imports) in the LLM prompt affect the compilability and correctness of generated tests compared to context-agnostic baselines?
- **RQ2:** Can LLM-generated tests achieve code coverage (line and branch coverage) comparable to that of manually written ground truth tests?
- **RQ3:** To what extent do the assertions in LLM-generated tests semantically align with the developer's original intent?

3 Scope & Assumptions

3.1 In Scope

- **Single-Method Unit Testing:** We focus exclusively on generating tests for individual "focal methods" isolated from the larger system context.
- **Languages:** The study targets **Java** (utilizing the Methods2Test dataset) and/or **Python** (utilizing the provided Zenodo dataset).
- **Empirical Evaluation:** We will automate the execution of generated tests to systematically measure compilability and code coverage.

3.2 Out of Scope

- **Integration/System Testing:** The generation of tests requiring external environments (e.g., databases, networks) is excluded.
- **GUI Testing:** Frontend and user interface automation are outside the scope of this project.
- **Complex Mocking:** While basic mocking may be employed, advanced dependency injection for highly coupled legacy code is not a primary focus.

3.3 Constraints

- **Data Availability:** The research relies on the quality and structure of existing public datasets (Methods2Test/Zenodo).
- **Compute Resources:** Experiments will be conducted using available LLM APIs (e.g., OpenAI, Anthropic, or open weights via Ollama), subject to rate limits and computational constraints.

4 Initial Methodology (High Level)

4.1 Datasets

We will utilize the **Methods2Test** dataset (for Java) or the equivalent **Python focal-method dataset** hosted on Zenodo. These datasets provide aligned pairs of focal methods and developer-written test cases, which will serve as the ground truth for our evaluation.

4.2 Tools & Techniques

- **LLMs:** We intend to leverage state-of-the-art models such as GPT-4o or CodeLlama, which are instruct-tuned for code generation tasks.
- **Prompt Engineering:** We will systematically design prompts that supply the LLM with the focal method body, its signature, and relevant surrounding context (e.g., class fields, imports) to mitigate hallucinations.
- **Test Runners:** Standard testing frameworks, such as JUnit (for Java) or PyTest (for Python), will be used to execute the generated tests and collect coverage metrics.

4.3 Analysis Plan

Our analysis will be **comparative** and **quantitative**:

1. **Generation Phase:** We will generate unit tests for a statistically significant subset of the dataset using our proposed LLM pipeline.
2. **Execution Phase:** Generated tests will be executed to assess their compilability and pass rate.
3. **Evaluation Phase:**
    - **Code Coverage:** We will calculate line and branch coverage percentages.
    - **Assertion Similarity:** We will measure the semantic similarity of generated assertions against the ground truth.
    - **Failure Analysis:** We will categorize failure modes (e.g., syntax errors, hallucinated dependencies) to identify systematic weaknesses.

5 Expected Outcomes

- **Software Artifact:** A prototype Command Line Interface (CLI) tool that accepts a method validation and outputs a functional unit test file.
- **Empirical Findings:** A comprehensive report detailing the success rates, coverage metrics, and identified failure patterns of LLM-generated tests.
- **Educational Value:** An enhanced understanding of the limitations of LLMs in strict syntactic environments and the intricacies of automated software testing.

6 Anticipated Risks

- **Non-determinism:** LLMs may produce stochastic outputs for identical inputs, complicating reproducibility. We will mitigate this by setting low temperature values where possible.
- **Environment Setup:** Effectively isolating tests for execution without full project context (the "missing dependency" problem) presents a significant technical challenge.